Lec-9

19/06/25 10:11 AM

String

S1 = "Hello"
S2 = "Hello"

S1 == S2

S1.equals(S2)

S1.compareTo(S2)

S1 - S2

Substring

Coding
0 1 2 3 4 5

c → [0,1]
co → [0,2]
Cod → [0,3]
Codi [0,4]
codin [0,5]
coding [0,6]

o [1,2]
od [1,3]
odi [1,4]
odin [1,5]
oding [1,6]

d [2,3]
di [2,4]
din [2,5]
ding [2,6]

i [3,4]
in [3,5]
ing [3,6]

n 4,5
ng 4,6

g 5,6

Substring(i,j)

for(i=0 i< s.len() i++){
for(j=i+1 j<=s.len j++){

}
}

0 1 2 3 4 5
coding

i j
0-1 c
1-2 o
2-3 d
3-4 i
4-5 n
5-6 g
j-i = 1
len=1
j-len = 0
2-1 = 1
3-1 = 2
4-1 = 3

0-2 co
1-3 od
2-4 di
3-5 in
4-6 ng
j-i=2
len=2
j-i=len
i = j-len

0-3 cod
1-4 odi
2-5 din
3-6 ing
j-i=3
len=3
j-len
=3-3=0
3-2=1

0-4 Codi
1-5 odin
2-6 ding
len=4

0-5 codin
1-6 oding
len=5

0-6 coding
len=6

for(len=1 len<=s.len() len++){
for(j=len j<=s.len, j++){
i = j-len

}
}

```
String s = " the sky is blue ";
```

trim()

the | sky | is | blue

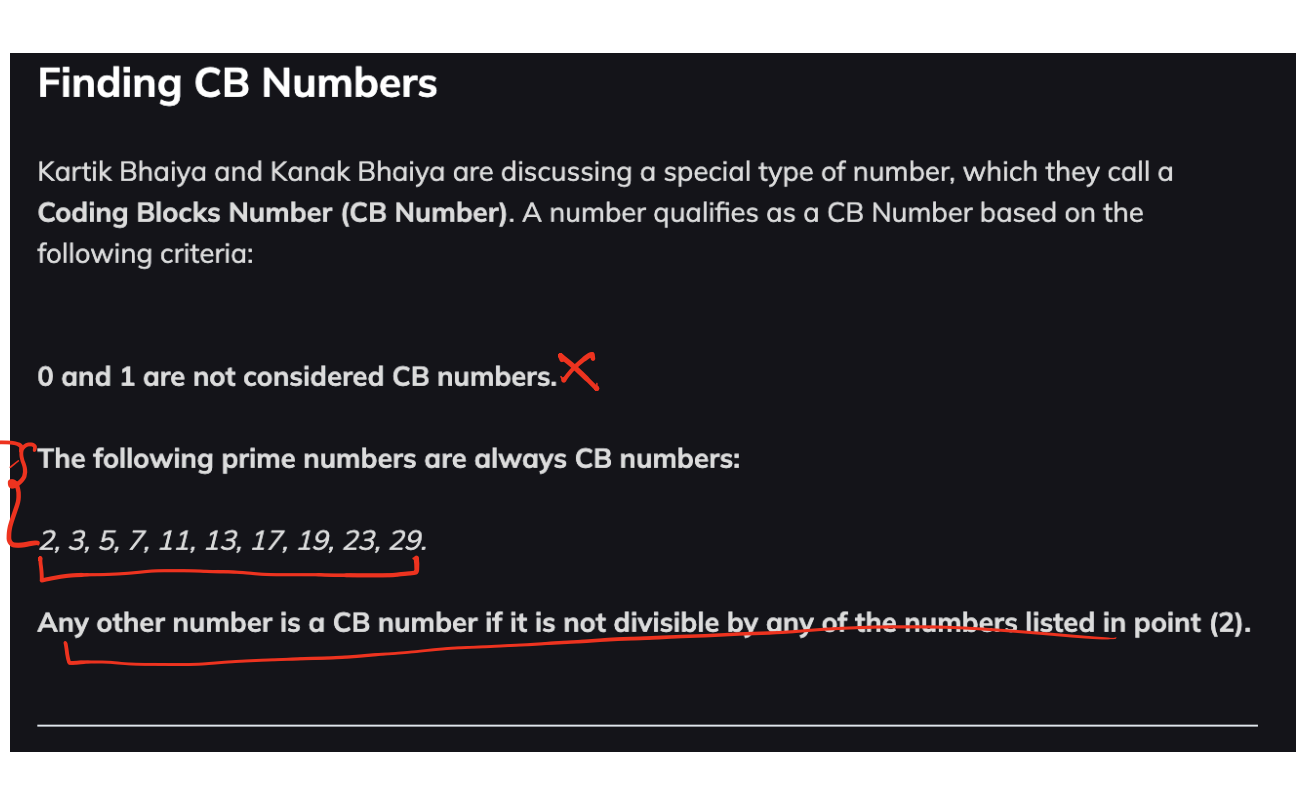
split(" ")

blue is sky the

kpcdaealmnoiapaiouokmpaa

ans=1 2 3 4

for(i=0 i<s.len() i++){
char ch= s.charAt(i)
if(isvowel(ch)){
c++
ans = max(ans, c)
}
else{
c = 0
}
}



Finding CB Numbers

Kartik Bhaiya and Kanak Bhaiya are discussing a special type of number, which they call a **Coding Blocks Number (CB Number)**. A number qualifies as a CB Number based on the following criteria:

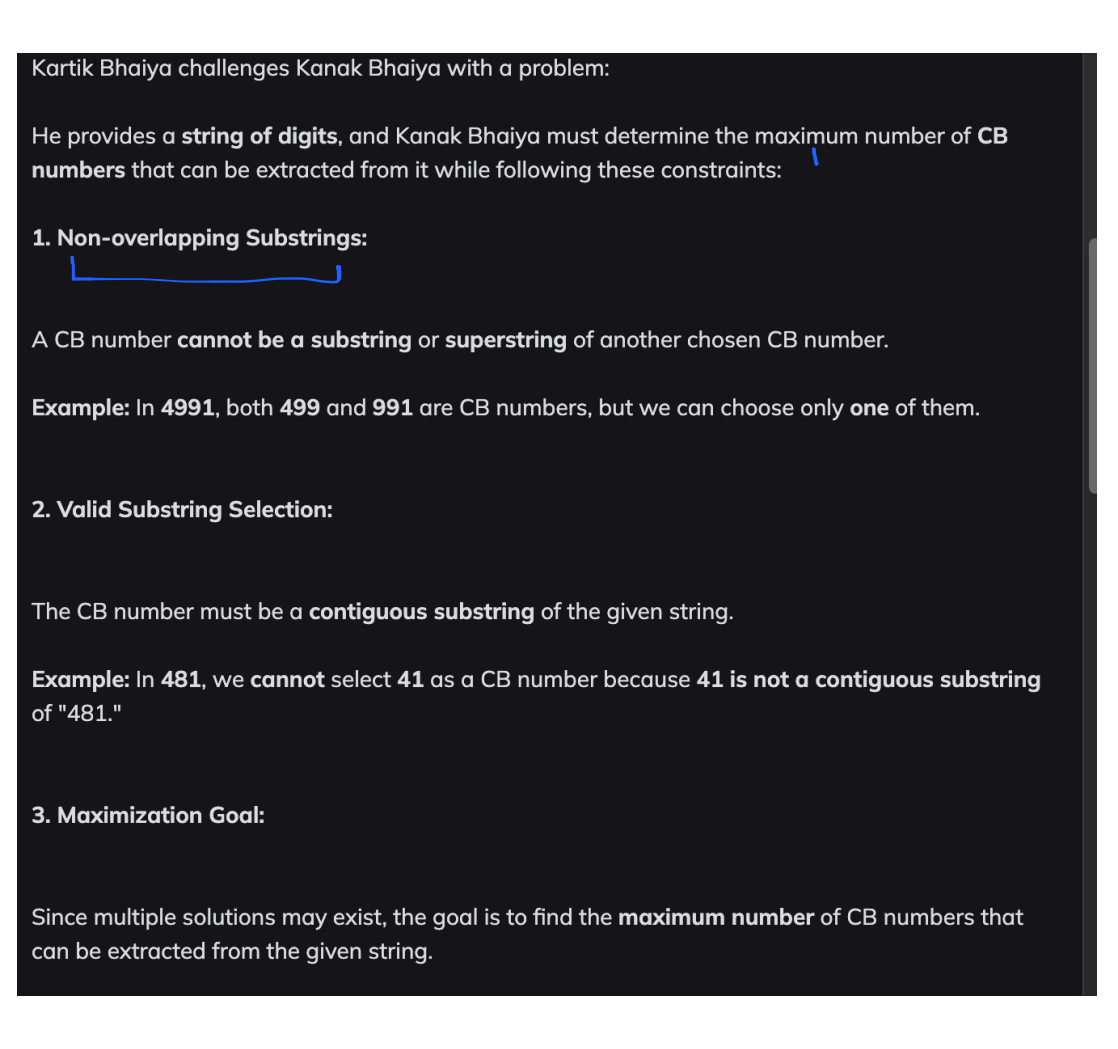
0 and 1 are not considered CB numbers.

The following prime numbers are always CB numbers:

2, 3, 5, 7, 11, 13, 17, 19, 23, 29.

Any other number is a CB number if it is not divisible by any of the numbers listed in point (2).

637



Kartik Bhaiya challenges Kanak Bhaiya with a problem:

He provides a **string of digits**, and Kanak Bhaiya must determine the maximum number of **CB numbers** that can be extracted from it while following these constraints:

1. Non-overlapping Substrings:

A CB number **cannot be a substring** or **superstring** of another chosen CB number.

Example: In **4991**, both **499** and **991** are CB numbers, but we can choose only **one** of them.

2. Valid Substring Selection:

The CB number must be a **contiguous substring** of the given string.

Example: In **481**, we **cannot** select **41** as a CB number because **41 is not a contiguous substring** of "481."

3. Maximization Goal:

Since multiple solutions may exist, the goal is to find the **maximum number** of CB numbers that can be extracted from the given string.

CB number

81635

4991

499 991

481 41

81615

8 → ✗
81 → ✗
816 → ✗
8161 → ✓
81615 ✗

1 ✗
16 ✗
161 ✗
1615 ✗

6 ✗
61
615 ✗

1 ✗
15 ✗

5

2

127

1 ✗
12 ✗
127 ✓

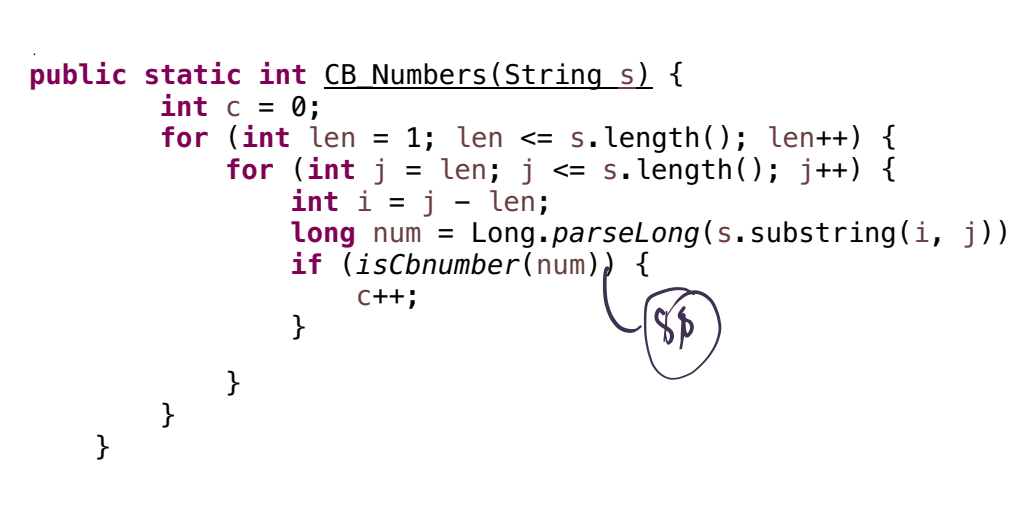
2 ✗
27 ✗

7 ✗

1
2
7

127
27

127



```
public static int CB_Numbers(String s) {
        int c = 0;
        for (int len = 1; len <= s.length(); len++) {
            for (int j = len; j <= s.length(); j++) {
                int i = j - len;
                long num = Long.parseLong(s.substring(i, j));
                if (isCbnumber(num)) {
                    c++;
                }

            }
        }
    }
```

→ [1,4]

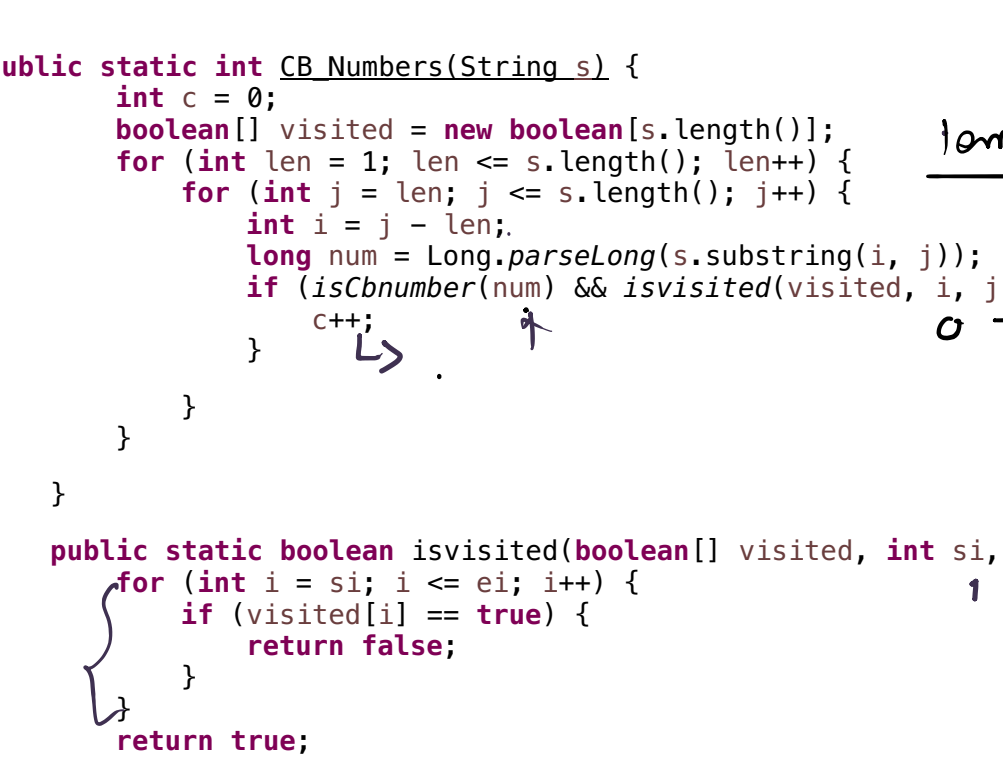
88

81615
01234

1 3

0 1 2 3 4
8 1 6 1 5

i, j-1



```
public static int CB_Numbers(String s) {
        int c = 0;
        boolean[] visited = new boolean[s.length()];
        for (int len = 1; len <= s.length(); len++) {
            for (int j = len; j <= s.length(); j++) {
                int i = j - len;
                long num = Long.parseLong(s.substring(i, j));
                if (isCbnumber(num) && isvisited(visited, i, j - 1)) {
                    c++;
                }

            }
        }

    }

    public static boolean isvisited(boolean[] visited, int si, int ei) {
        for (int i = si; i <= ei; i++) {
            if (visited[i] == true) {
                return false;
            }
        }
        return true;

    }
```

len=3

1 2 7
0 1 2

i = j-len
= 0

0 2

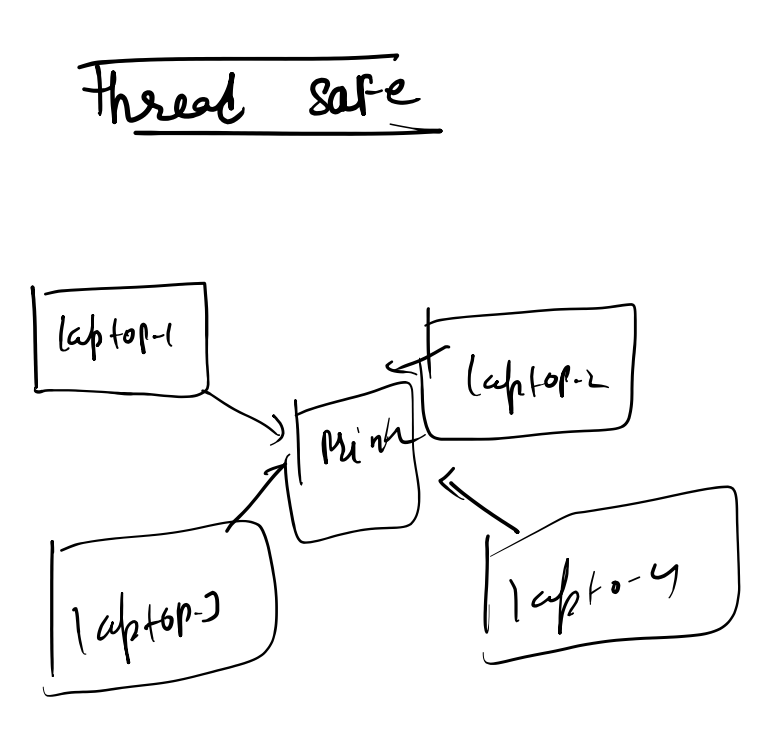
T T
0 1 2

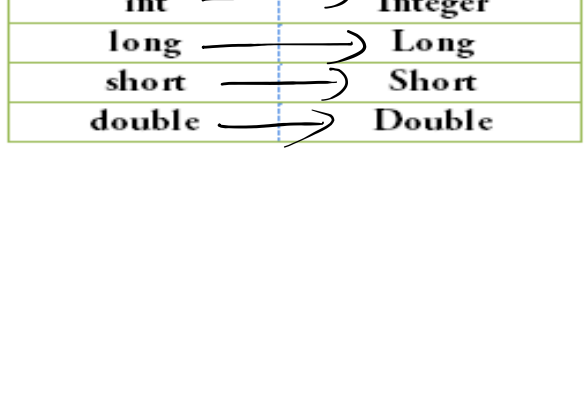
c = 1 2

16 x 2 + 2 = 34

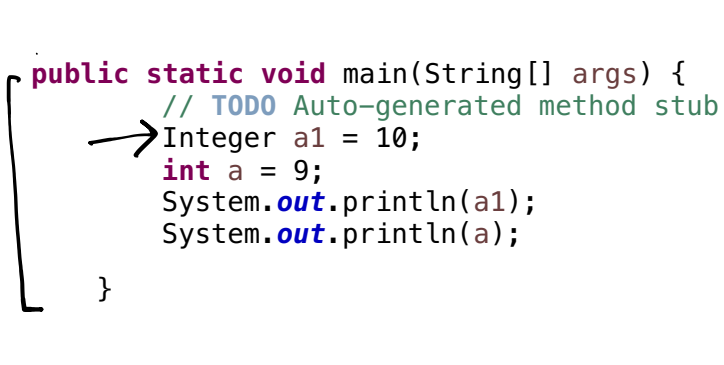
oldcap x 2 + 2

Thread safe





Primitive type	Wrapper Class
boolean	Boolean
byte	Byte
char	Character
float	Float
int	Integer
long	Long
short	Short
double	Double



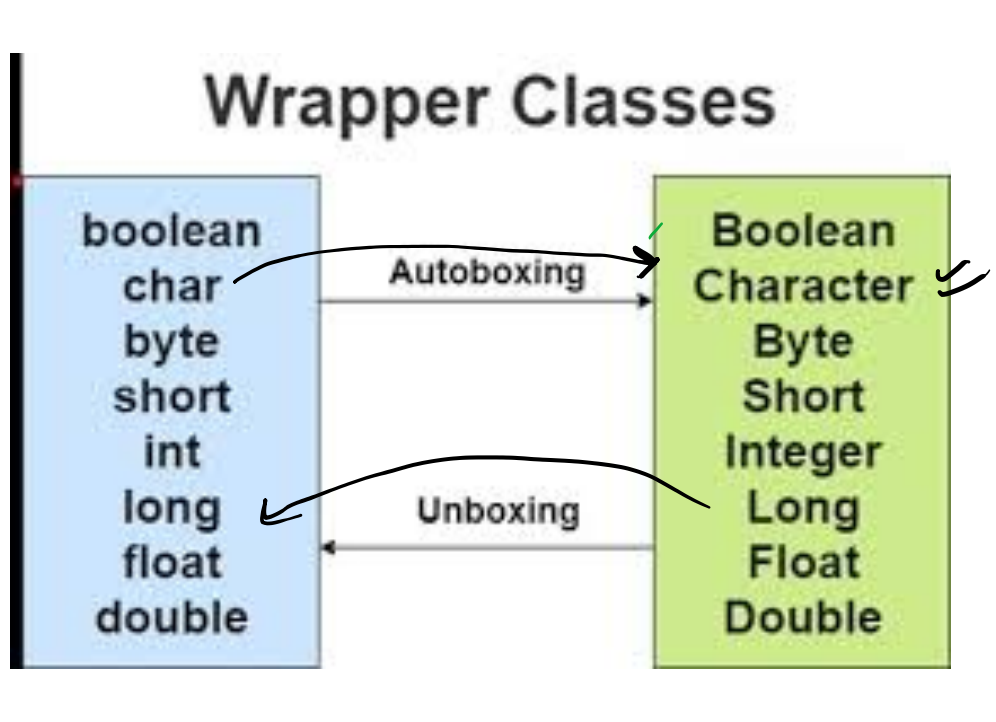
```
public static void main(String[] args) {
        // TODO Auto-generated method stub
        Integer a1 = 10;
        int a = 9;
        System.out.println(a1);
        System.out.println(a);

    }
```

main

a1 = 2K
a = 9

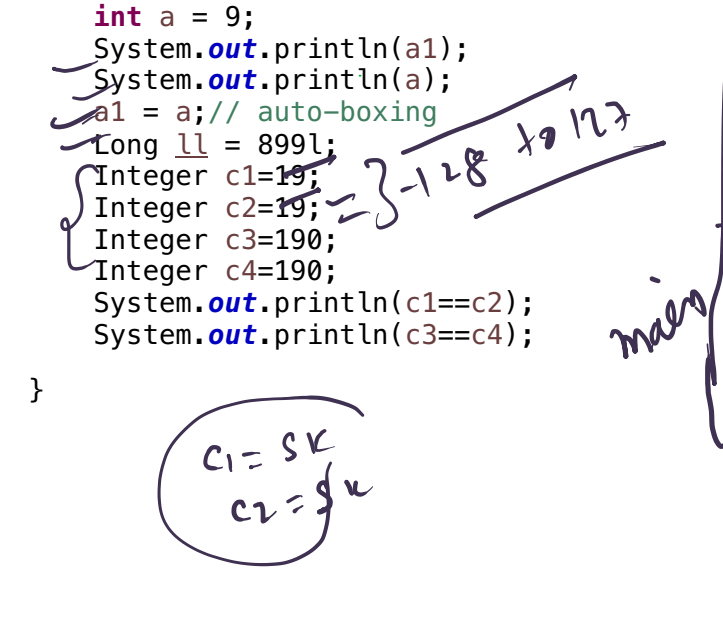
Heap

10
2K



Wrapper Classes

Stack → Heap



```
public static void main(String[] args) {
        // TODO Auto-generated method stub
        Integer a1 = 10;
        int a = 9;
        System.out.println(a1);
        System.out.println(a);
        a1 = a;// auto-boxing
        Long ll = 899l;
        Integer c1=19;
        Integer c2=19;
        Integer c3=190;
        Integer c4=190;
        System.out.println(c1==c2);
        System.out.println(c3==c4);

    }
```

-128 to 127

c1 = 5K
c2 = 5K

main

a1 = 2K
a = 9

10
2K

cache → size → 256

-128 -127 - -
19
5K
127
0 1 2 3 4 5 6 - - -
255

Generics < >

① add
② remove
③ get
④ update
⑤ Sort
⑥ display

0 1 2 3
[10, 20, 30, 40]

(0,-9) [-9, 10, 20, 30, 40]
(1,-9) [10, -9, 20, 30, 40]
(2,-9) [10, 20, -9, 30, 40]
(3,-9) [10, 20, 30, -9, 90]
(4,-9) [10, 20, 30, 40, -9]

10 →

15 x 1.5 = 22

22

0 1 2 3 4 5 6 7 8 9 10

"JIS RAASTE PE HUM CHAL RAHE HAI WOH HAI ONE WAY, NA REVERSE GEAR HAI NA U-TURN"

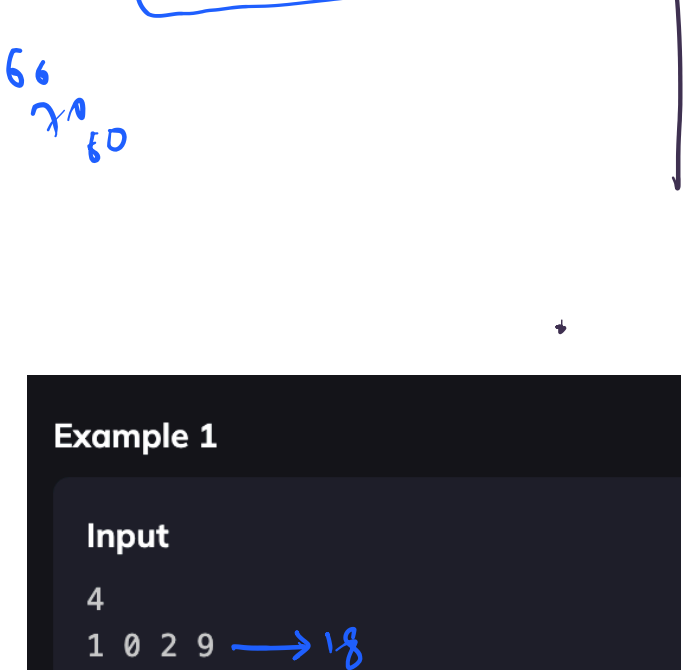
For

[10, 20, 30, 40]

2x

10 → 10 — 1
20 → 10 20 — 2
30 → 10 20 30 40 — 2+1, 1
40
50 → 10 20 30 40 50 60 70 80 — 4+1, 3
60
70
80

8 → 15
1 → 15/8 ≈ 2



Example 1

Input

```
4
1 0 2 9
5
3 4 5 6 7
```

Output

```
3, 5, 5, 9, 6, END
```

18
18

0 0 1
1 0 2 9
3 4 5 6 7
3 5 5 9 6

18
1029
34567

1
1 8 9
3 4
1 2 3

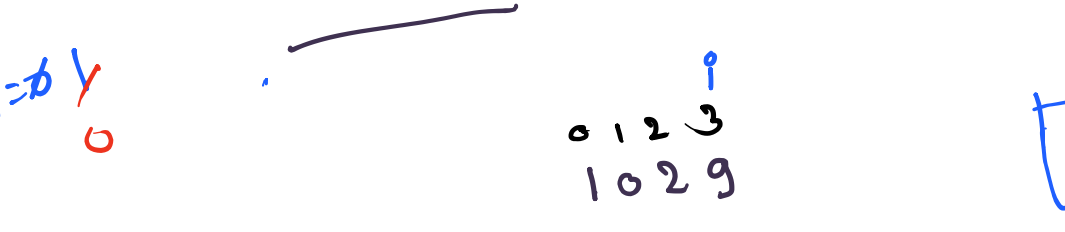
carry = 0 1
0

1
[2, 3]
[5 7]
9 0

1 0 2 9 → 4
3 4 5 6 7 → 5
3 5 5 9 6 → 5

0 1 2 3
1 0 2 9
7 9 2 4 5 6 7
0 1 2 3 4 5 6
7 9 2 5 5 9 6

6 9 5 5 2 9 7



```
public static void Sum_Of_Two_Arrays(int[] arr1, int[] arr2) {
        int i = arr1.length - 1;
        int j = arr2.length - 1;
        ArrayList<Integer> ll = new ArrayList<>();
        int carry = 0;
        while (i >= 0 && j >= 0) {
            int sum = arr1[i] + arr2[j] + carry;
            ll.add(sum % 10);
            carry = sum / 10;
            i--;
            j--;
        }

    }
```

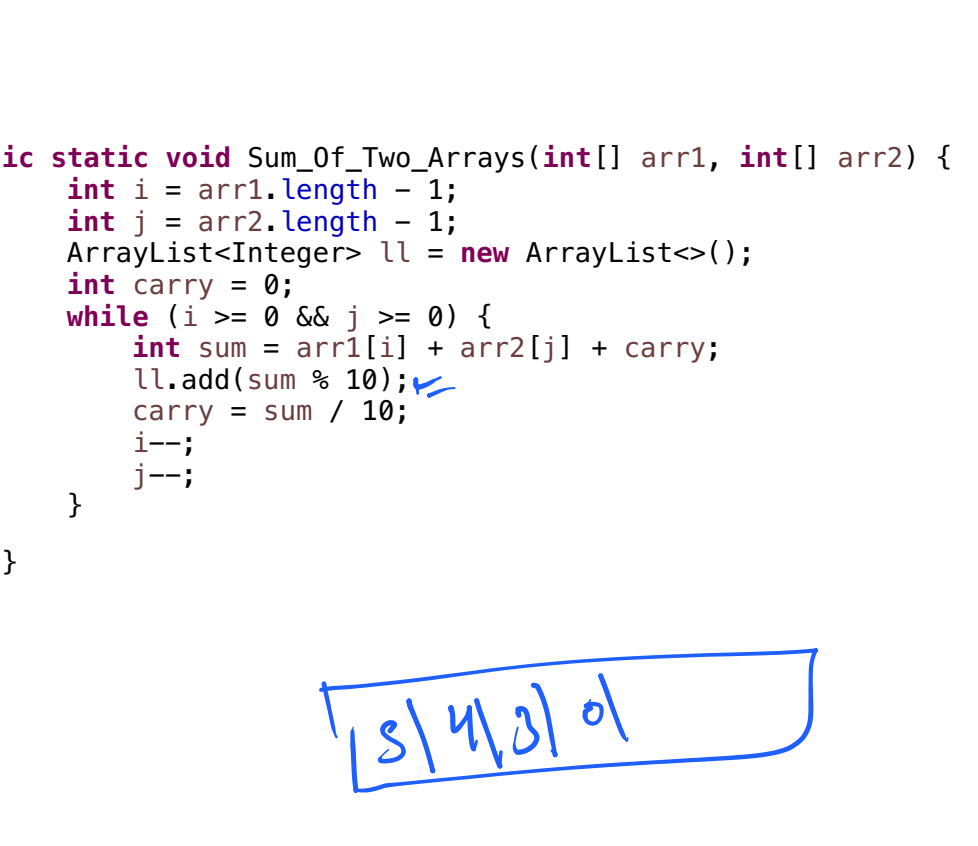
1 1 0 2 4

c = 0
1

5 7 9 3 2 1

5 4 3 0

0 + 4 + 1 = 5



```
public static void Sum_Of_Two_Arrays(int[] arr1, int[] arr2) {
        int i = arr1.length - 1;
        int j = arr2.length - 1;
        ArrayList<Integer> ll = new ArrayList<>();
        int carry = 0;
        while (i >= 0 && j >= 0) {
            int sum = arr1[i] + arr2[j] + carry;
            ll.add(sum % 10);
            carry = sum / 10;
            i--;
            j--;
        }
        while (i >= 0) {
            int sum = arr1[i]+ carry;
            ll.add(sum % 10);
            carry = sum / 10;
            i--;

        }
        while ( j >= 0) {
            int sum = arr2[j] + carry;
            ll.add(sum % 10);
            carry = sum / 10;
            j--;
        }

    }
```

98
89
187

9 8
8 9

17

7 8 1